

Enhanced ER Modeling

Chapter # 04

Enhanced ER (EER) model

Created to design more accurate database schemas

- Reflect the data properties and constraints more precisely

More complex requirements than traditional applications

Enhanced ER (EER) model

EER model includes all modeling concepts of the ER model

In addition, EER includes:

- **Subclasses** and **super classes**
- **Specialization** and **generalization**
- **Category** or **union type**
- **Attribute** and **relationship inheritance**

Subclasses and Super classes

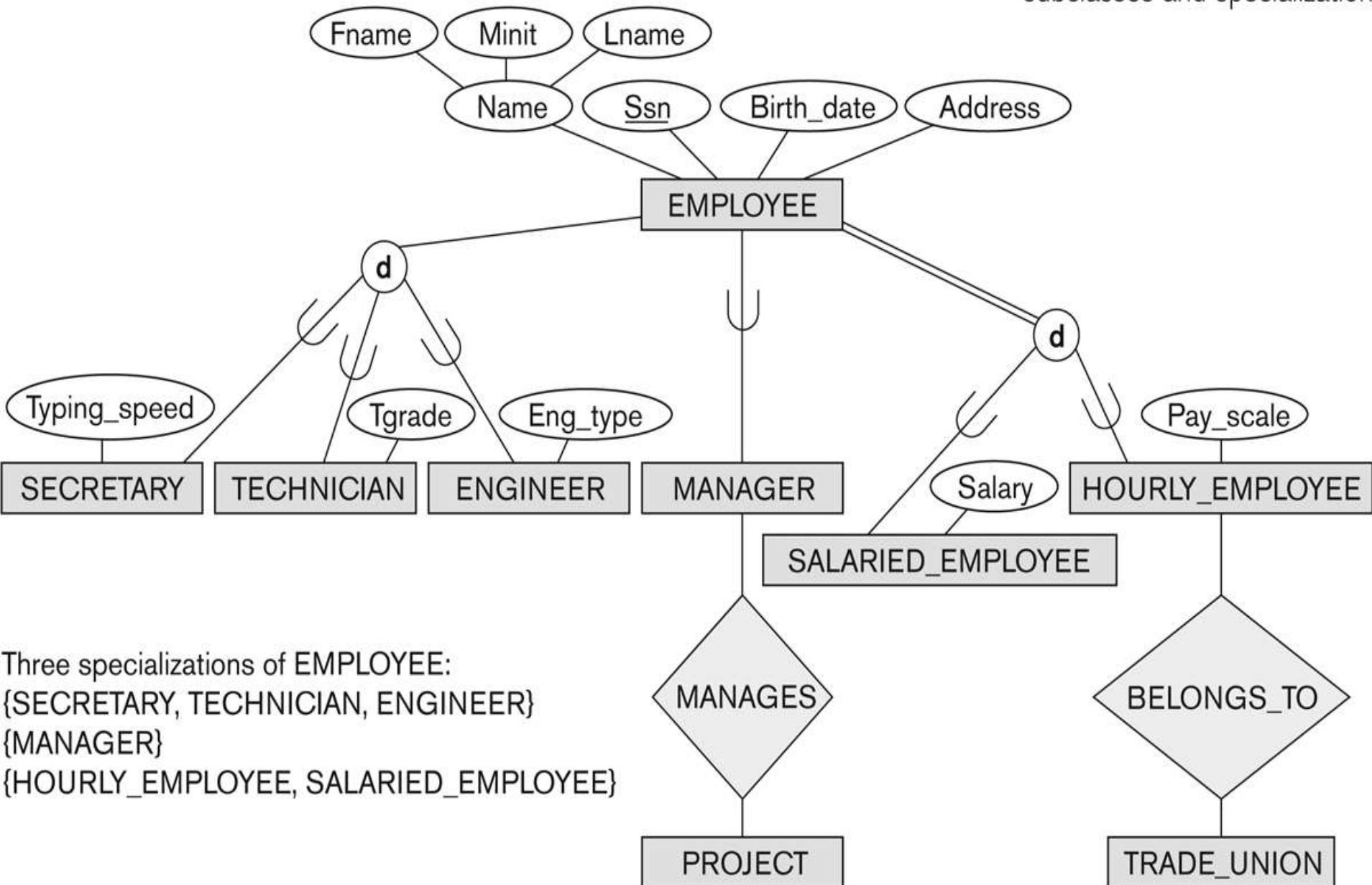
In many cases an entity type has numerous sub groupings of its entities that are meaningful and need to be represented explicitly because of their significance to the database application.

For example: **EMPLOYEE** may be further grouped into:

- SECRETARY, ENGINEER, TECHNICIAN, ...
 - Based on the EMPLOYEE's Job
- MANAGER
 - EMPLOYEEs who are managers
- SALARIED_EMPLOYEE, HOURLY_EMPLOYEE
 - Based on the EMPLOYEE's method of pay

Figure 4.1

EER diagram notation to represent subclasses and specialization.



Subclasses and Super classes

We call each of these sub groupings a **subclass** of the EMPLOYEE entity type, and the EMPLOYEE entity type is called the **super class** for each of these subclasses.

These are called **super class/subclass** (as well as simply **class/subclass**) relationships:

- EMPLOYEE/SECRETARY
- EMPLOYEE/TECHNICIAN
- EMPLOYEE/MANAGER

These are also called **IS-A** relationships

- SECRETARY IS-A EMPLOYEE, TECHNICIAN IS-A EMPLOYEE,

Subclasses and Super classes

- An Entity **CANNOT** exist in the database only by being a member of a subclass; it must also be a member of the superclass
- A member of the superclass can be optionally included as a member of any number of its subclasses

Subclasses and Super classes

A salaried employee who is also an engineer belongs to the two subclasses:

ENGINEER, and SALARIED_EMPLOYEE

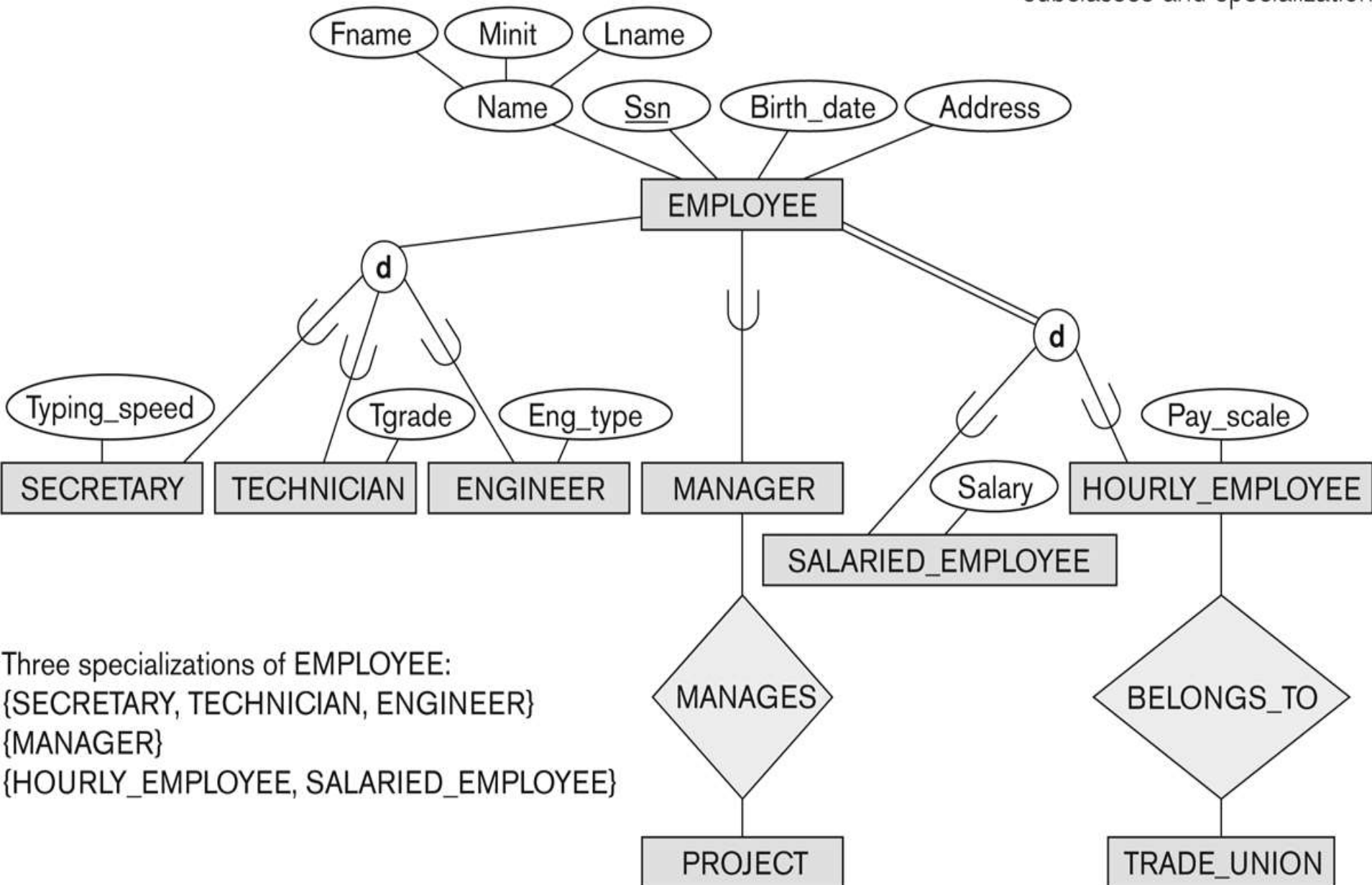
A salaried employee who is also an engineering manager belongs to the three subclasses:

MANAGER,ENGINEER, and SALARIED_EMPLOYEE

It is not necessary that every entity in a superclass be a member of some subclass

Figure 4.1

EER diagram notation to represent subclasses and specialization.



Subclasses, Superclasses and Inheritance

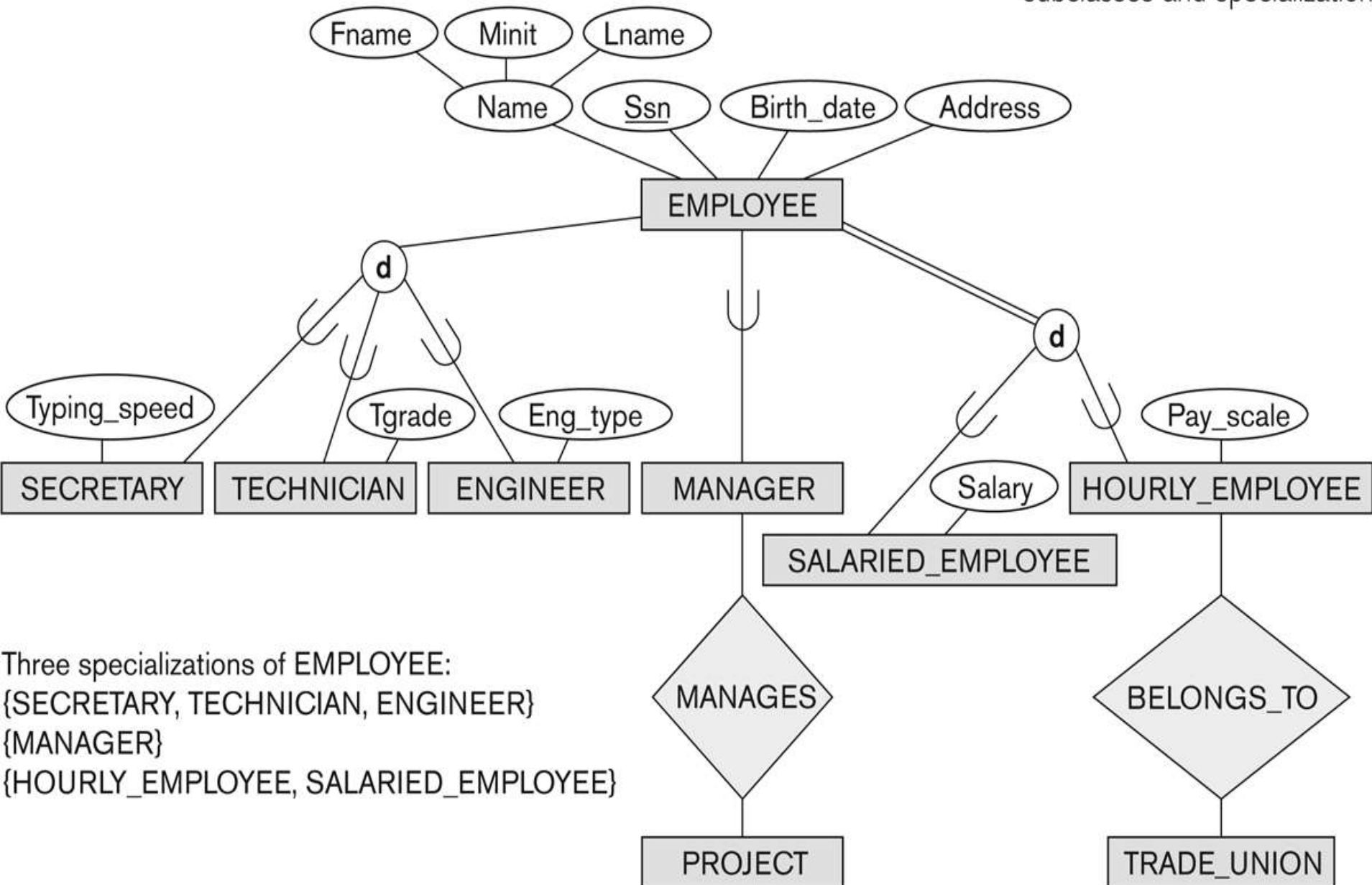
An important concept associated with subclasses is that of **Type inheritance**

An entity that is member of a subclass *inherits*

- *All attributes of the entity as a member of the superclass*
- *All relationships of the entity as a member of the superclass*

Figure 4.1

EER diagram notation to represent subclasses and specialization.



Subclasses, Superclasses and Inheritance

In the previous slide, SECRETARY (as well as TECHNICIAN and ENGINEER) inherit the attributes Name, SSN, ..., from EMPLOYEE

- Every SECRETARY entity will have values for the inherited attributes
- Every SECRETARY entity will also keep all relationships

Specialization

Specialization is the process of defining a set of subclasses of an entity type

The set of subclasses is based upon some distinguishing characteristics of the entities in the super class

Example:

{SECRETARY, ENGINEER, TECHNICIAN} is a specialization of EMPLOYEE based upon ***job type***.

Specialization

There may be several specializations of the same super class

Example: Another specialization of EMPLOYEE based on **method of pay** is {SALARIED_EMPLOYEE, HOURLY_EMPLOYEE}.

The subset symbol on each line connecting a subclass to the circle indicates the direction of the super class/subclass relationship.

Specialization

Attributes of a subclass are called *specific* or *local* attributes.

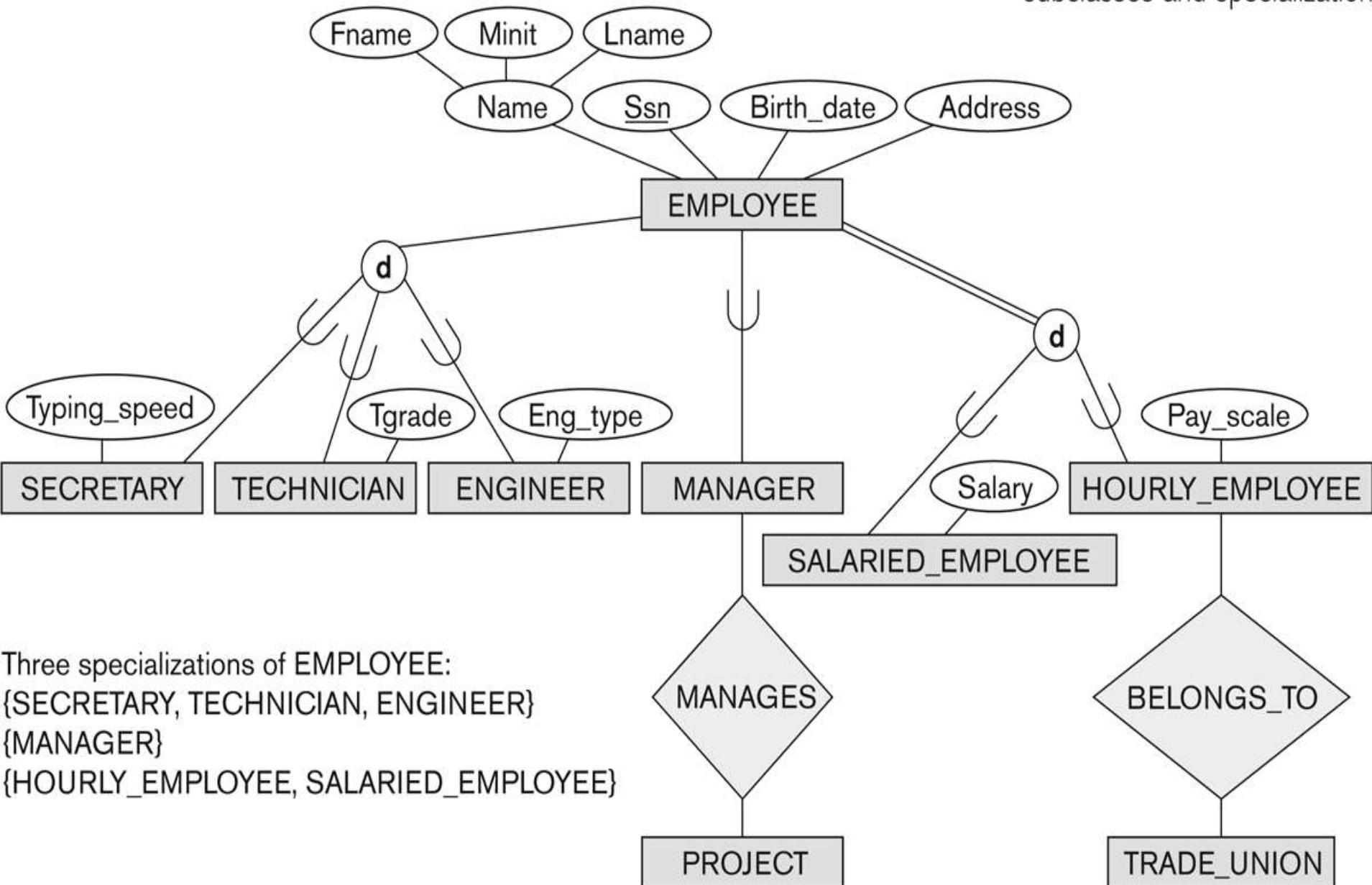
- For example, the attribute Typing Speed of SECRETARY

The subclass can also participate in specific relationship types

For example, a relationship BELONGS_TO HOURLY_EMPLOYEE

Figure 4.1

EER diagram notation to represent subclasses and specialization.



Specialization

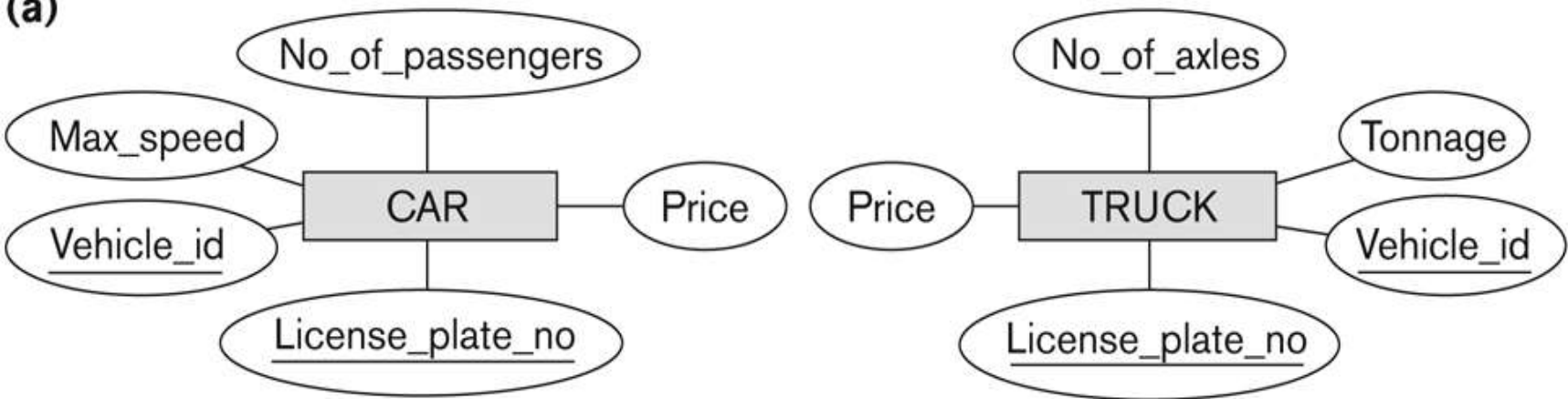
In summary, the specialization process allows us to do the following

- Define a set of subclass of an entity type
- Establish additional specific attributes with each subclass
- Establish additional specific relationship types between each subclass and other entity types or other subclasses

Generalization

- The reverse of the specialization process
- Several classes with common features are generalized into a superclass; original classes become its subclasses
- Example: CAR, TRUCK generalized into VEHICLE; both CAR, TRUCK become subclasses of the superclass VEHICLE.
 - We can view {CAR, TRUCK} as a specialization of VEHICLE
 - Alternatively, we can view VEHICLE as a generalization of CAR and TRUCK

(a)



(b)

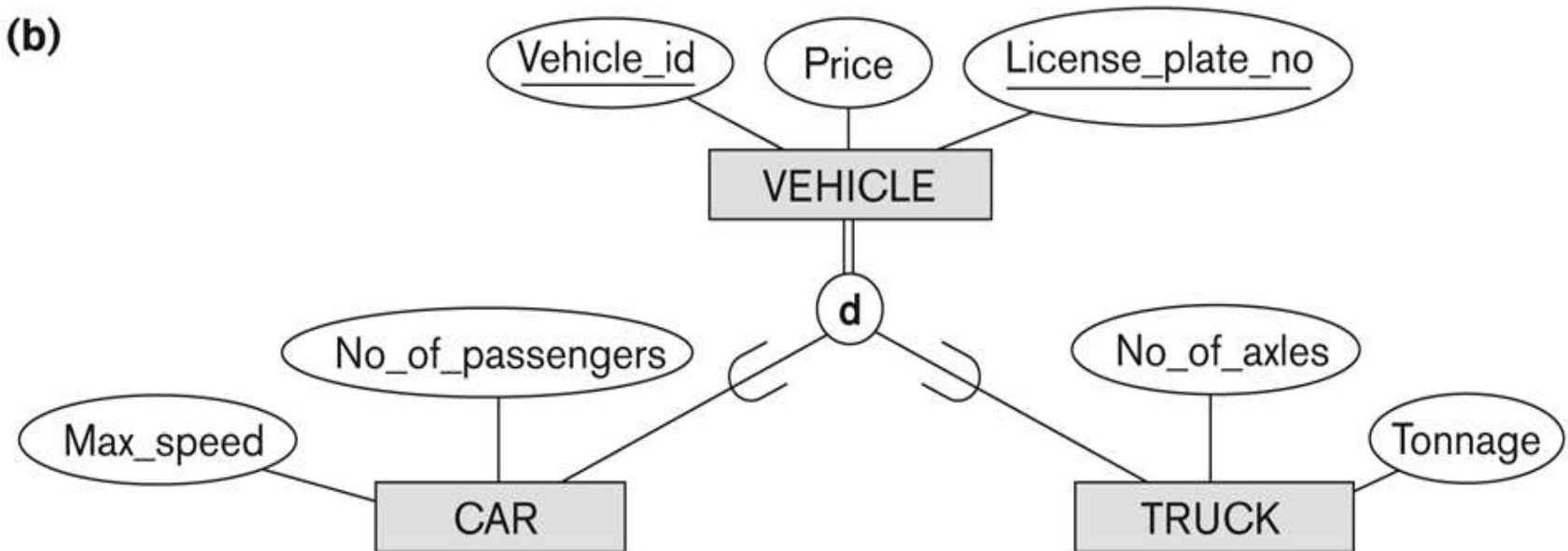


Figure 4.3

Generalization. (a) Two entity types, CAR and TRUCK. (b) Generalizing CAR and TRUCK into the superclass VEHICLE.

Generalization

Diagrammatic notation sometimes used to distinguish between generalization and specialization

- Arrow pointing to the generalized super class represents a generalization
- Arrows pointing to the specialized subclasses represent a specialization
- We do not use this notation because it is often subjective as to which process is more appropriate for a particular situation

Generalization

Data Modeling with Specialization and Generalization

- A superclass or subclass represents a set of entities
- Shown in rectangles in EER diagrams (as are entity types)
- Sometimes, all entity sets are simply called classes, whether they are entity types, superclasses, or subclasses

Constraints on Specialization and Generalization

If we can determine exactly those entities that will become members of each subclass by a condition, the subclasses are called **predicate-defined** (or **condition-defined**) subclasses

- Condition is a constraint that determines subclass members
- Display a predicate-defined subclass by writing the predicate condition next to the line attaching the subclass to its superclass

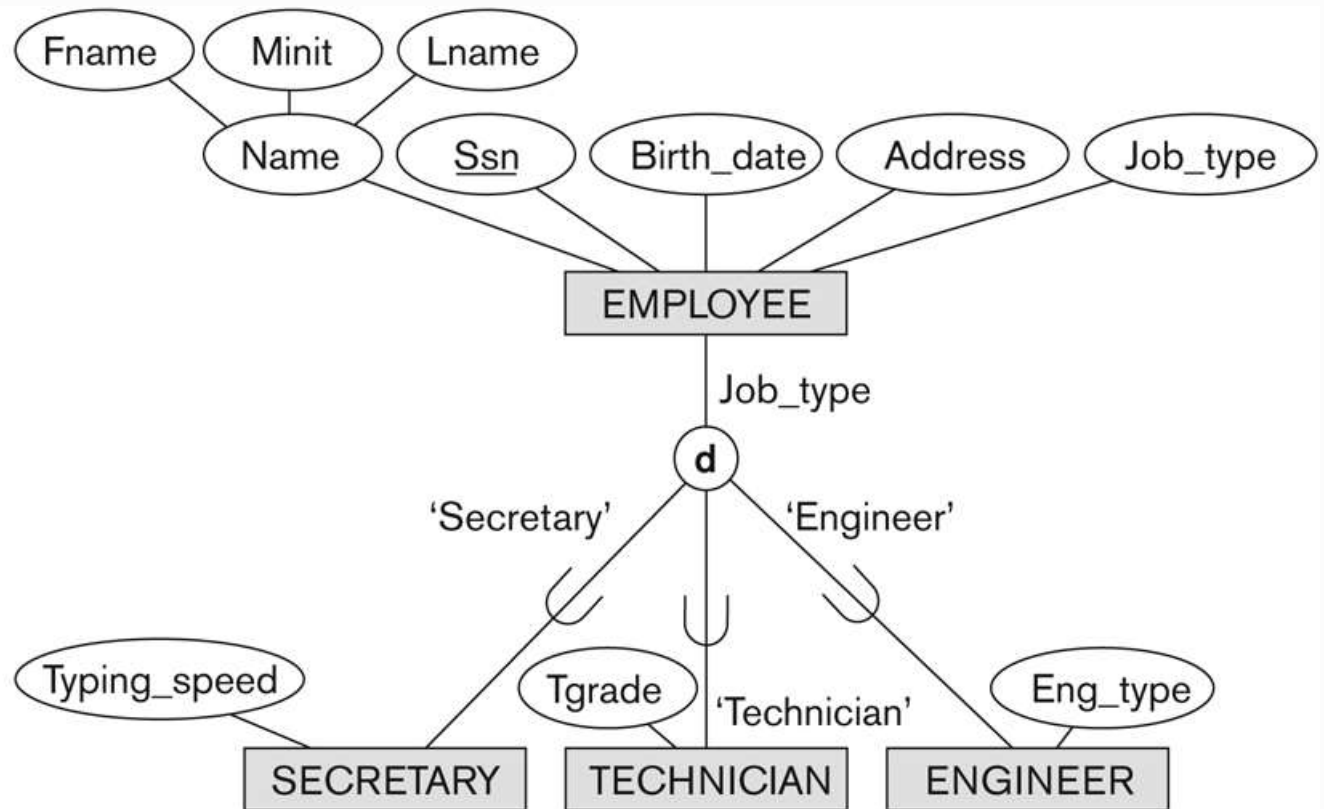
If all subclasses in a specialization have membership condition on same attribute of the superclass, specialization is called an **attribute defined-specialization**

- Attribute is called the defining attribute of the specialization
- Example: JobType is the defining attribute of the specialization {SECRETARY, TECHNICIAN, ENGINEER} of EMPLOYEE

Constraints on Specialization and Generalization

Figure 4.4

EER diagram notation for an attribute-defined specialization on Job_type.



Constraints on Specialization and Generalization

If no condition determines membership, the subclass is called *user-defined*

- Membership in a subclass is determined by the database users by applying an operation to add an entity to the subclass
- Membership in the subclass is specified individually for each entity in the superclass by the user

Constraints on Specialization and Generalization

Two basic constraints can apply to a specialization/generalization

- *Disjointness Constraint*
- *Completeness Constraint*

Disjointness Constraint

Specifies that the subclasses of the specialization must be disjoint
(an entity can be a member of at most one of the subclasses of the specialization)

Specified by **d** in EER diagram

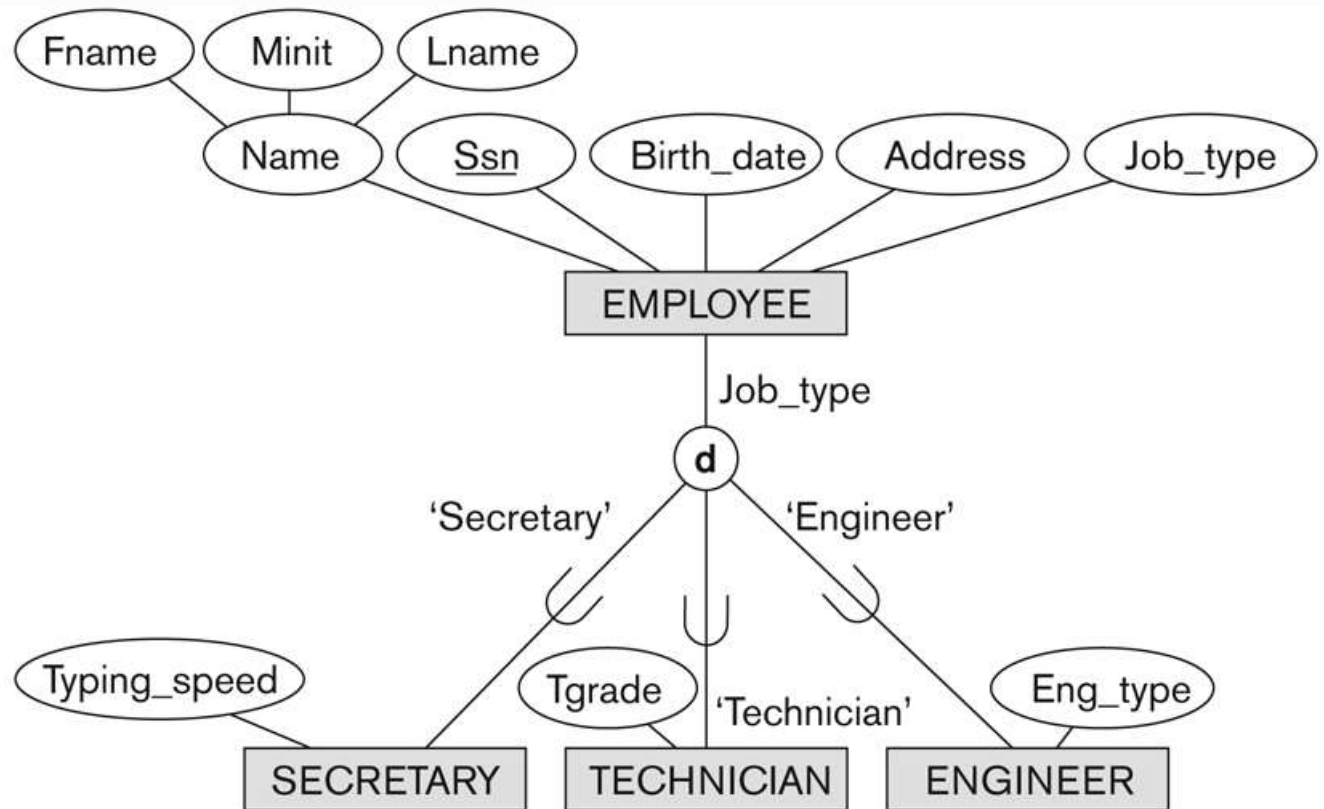
If not disjointed, overlap; that is the same entity may be a member of more than one subclass of the specialization

Specified by **o** in EER diagram

Disjointness Constraint

Figure 4.4

EER diagram notation for an attribute-defined specialization on Job_type.



Overlapping Constraint

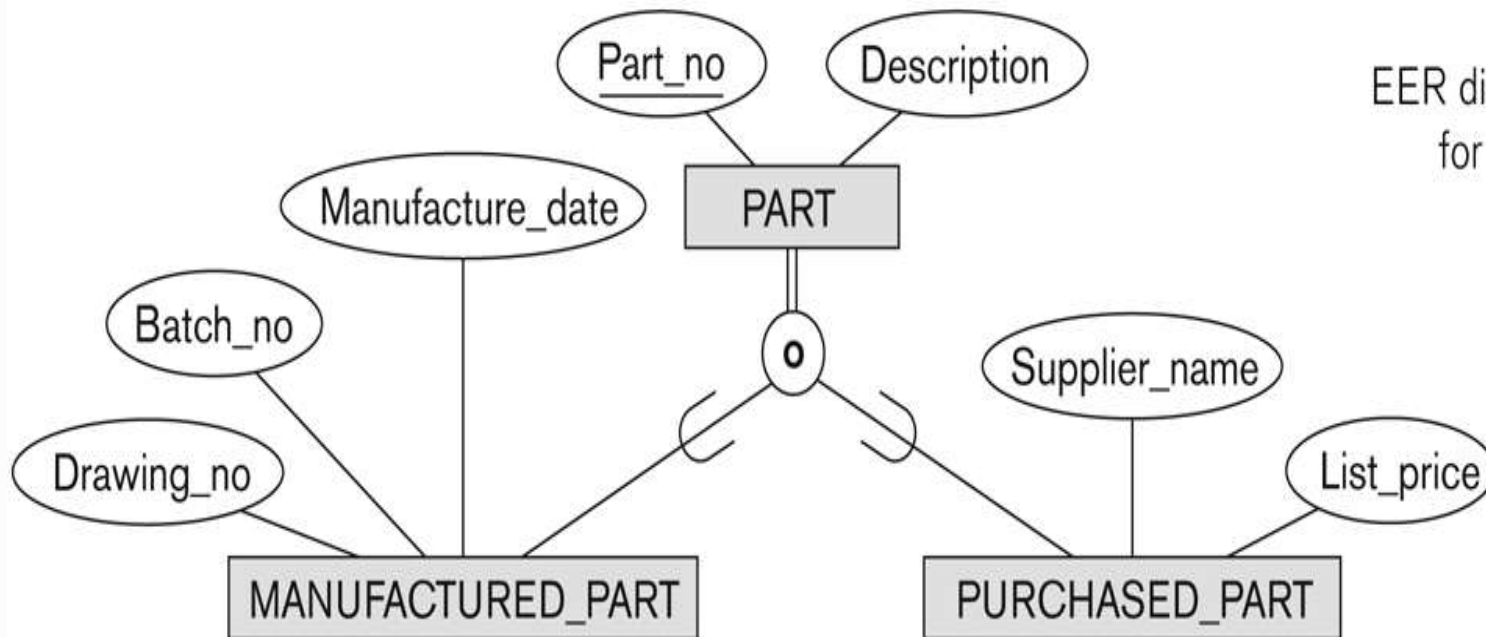


Figure 4.5

EER diagram notation
for an overlapping
(nondisjoint)
specialization.

Completeness Constraint

Completeness Constraint

Total specifies that every entity in the super class must be a member of some subclass in the specialization/ generalization

Shown in EER diagrams by a **double line**

For example, if every EMPLOYEE must be either an HOURLY_EMPLOYEE or a SALARIED_EMPLOYEE, then the specialization {HOURLY_EMPLOYEE,SALARIED_EMPLOYEE} in is a total specialization of EMPLOYEE

Partial allows an entity not to belong to any of the subclasses

Shown in EER diagrams by a **single line**

For example, if some EMPLOYEE entities do not belong to any of the subclasses {SECRETARY,ENGINEER,TECHNICIAN} in, then that specialization is partial

Completeness Constraint

The disjointness and completeness constraints are independent.
Hence, we have the following four possible constraints on specialization:

- Disjoint, total
- Disjoint, partial
- Overlapping, total
- Overlapping, partial

Constraints on Specialization and Generalization

Some general rules

- Deleting an entity from a superclass implies that it is automatically deleted from all the subclasses to which it belongs
- Inserting an entity in a superclass of a total specialization implies that the entity is mandatorily inserted in at least one of the subclasses of the specialization

Specialization / Generalization Hierarchies, Lattices and Shared Subclasses

- A subclass may itself have further subclasses specified on it
- Forms a hierarchy or a lattice
- Hierarchy has a constraint that every subclass has only one superclass (called **single inheritance**)
- In a lattice, a subclass can be subclass of more than one superclass (called **multiple inheritance**)
- In a lattice or hierarchy, a subclass inherits attributes not only of its direct superclass, but also of all its predecessor superclasses
- A subclass with more than one superclass is called a **shared subclass**

Specialization / Generalization Hierarchies, Lattices and Shared Subclasses

- In specialization, start with an entity type and then define subclasses of the entity type by successive specialization (top down conceptual refinement process)
- In generalization, start with many entity types and generalize those that have common properties (bottom up conceptual synthesis process)

Specialization Lattice

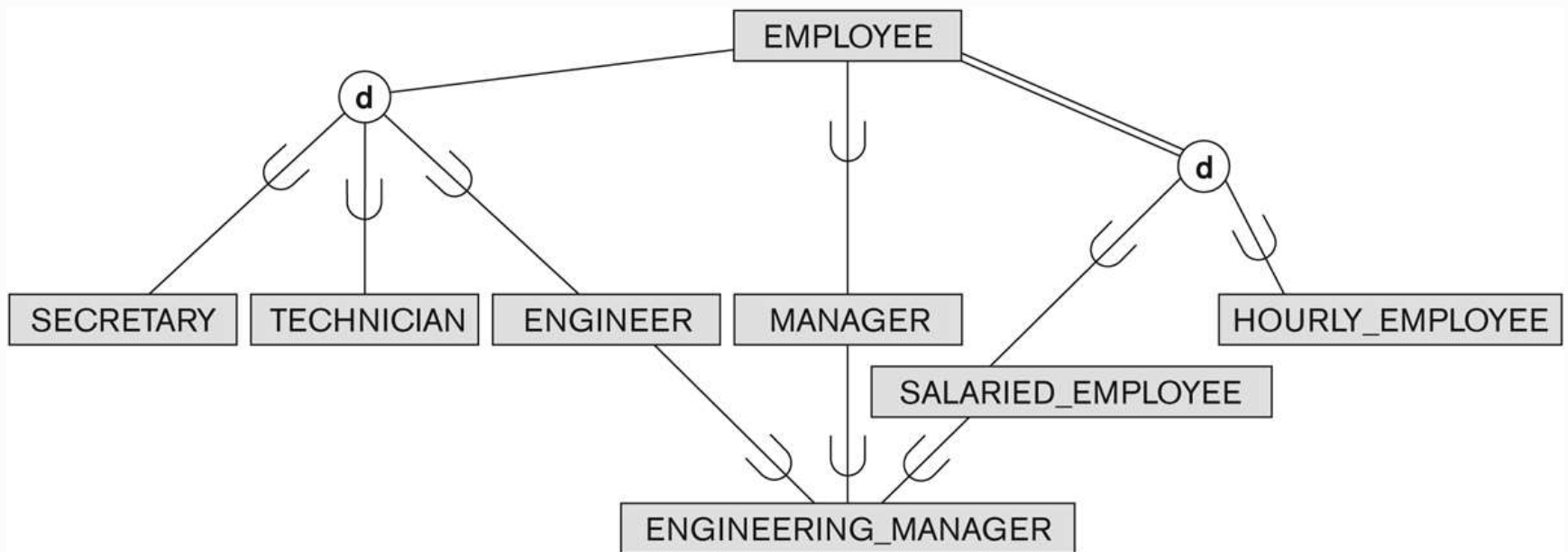
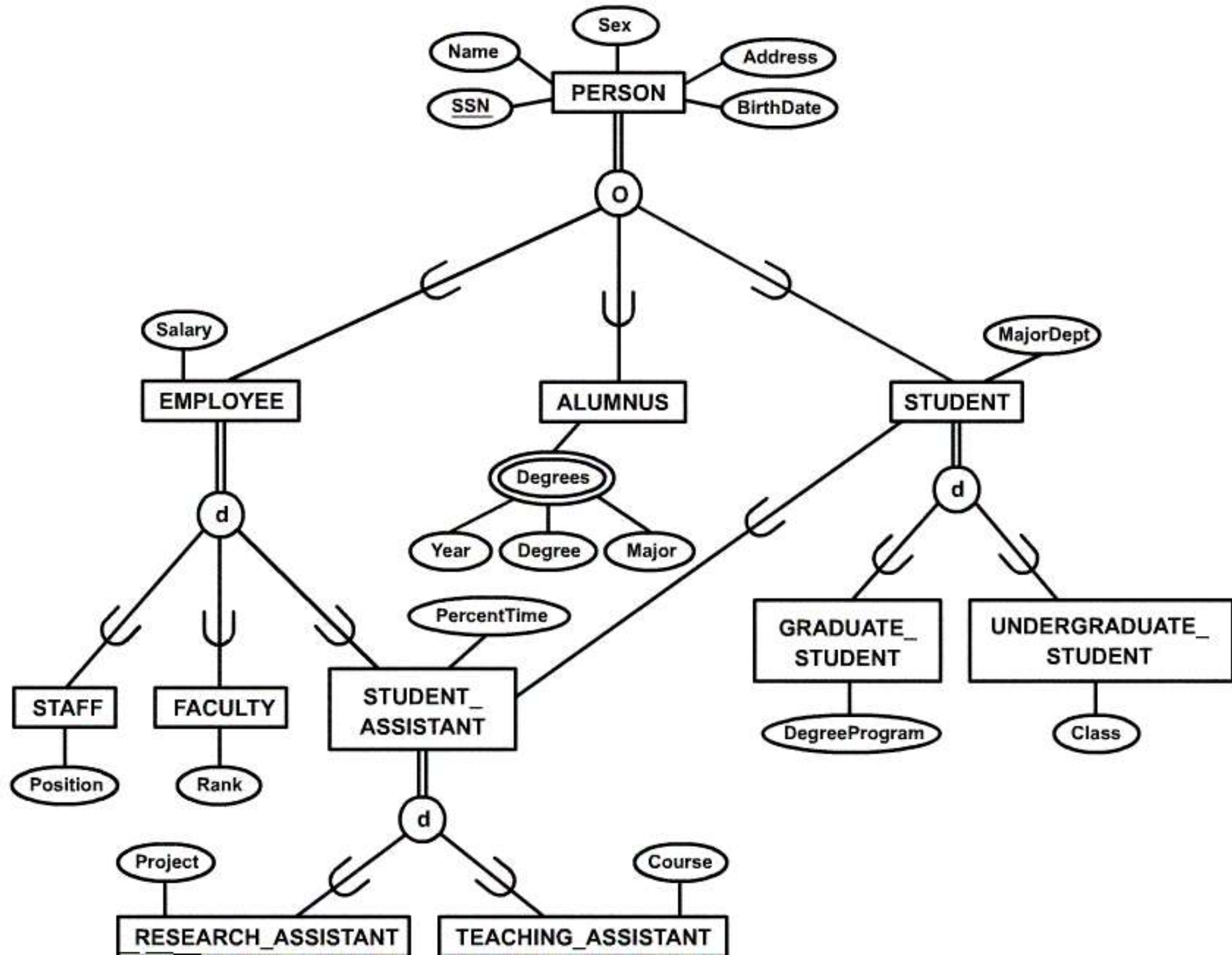


Figure 4.6

A specialization lattice with shared subclass ENGINEERING_MANAGER.



Categories (UNION TYPES)

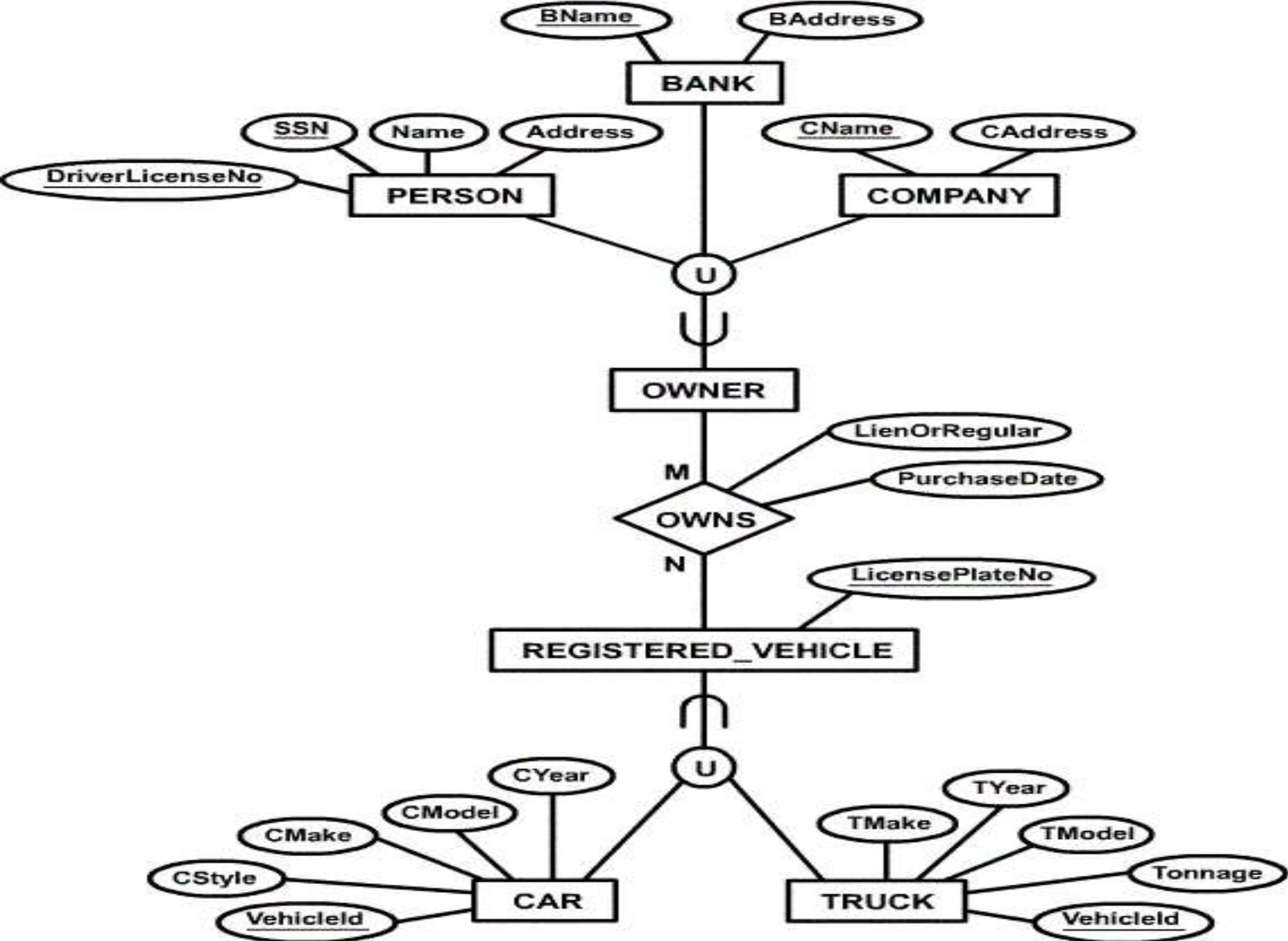
- All of the superclass/subclass relationships we have seen thus far have a single superclass
- A shared subclass is subclass in more than one distinct superclass/subclass relationships, where each relationship has a single superclass (multiple inheritance)
- In some cases, need to model a single superclass/subclass relationship with more than one superclass
- Superclasses represent different entity types
- Such a subclass is called a **category** or **UNION TYPE**

Categories (UNION TYPES)

Example: Database for vehicle registration, vehicle owner can be a person, a bank (holding a lien on a vehicle) or a company.

- Category (subclass) OWNER is a subset of the union of the three superclasses COMPANY, BANK, and PERSON
- A category member must exist in at least one of its superclasses

Note: The difference from shared subclass, which is subset of the intersection of its superclasses (shared subclass member must exist in all of its superclasses).



Categories (UNION TYPES)

- We can compare a UNION (OWNER) with shared subclass (ENGINEERING_MANAGER)
- The latter is a subclass of each of the three superclass ENGINEER, MANAGER and SALARIED_EMPLOYEE, so an entity that is a member of ENGINEERING_MANAGER **must exist in all three**
- This means that an engineering manager must be an ENGINEER, a MANAGER, and a SALARIED_EMPLOYEE
- On the other hand, an entity that is a member of OWNER must exist in **only one** of the superclass

Categories (UNION TYPES)

- Attribute inheritance works more selectively in the case of UNION.
- For example, OWNER entity inherits attributes of a COMPANY, a PERSON **OR** a BANK
- A shared subclass such as ENGINEERING_MANAGER inherits **ALL** the attributes of its superclasses